

INT202 Complexity of Algorithms
Tutorial 1

1. Find the asymptotic notation in Big-Oh

- (a)

```
int i = 1;
    i++;
    int m = i * i;
```
- (b)

```
int i = 1;
while (i < n){
    i = i * 2;
}
```
- (c)

```
int i = 2;
while (i < n){
    i = i * i;
}
```
- (d)

```
int j = 0;
for (int i = 0; i <= n; i++) {
    j = i;
    j = j * j;
}
```
- (e)

```
for (int m = 0; m < n; m++) {
    i = 1;
    while (i < n){
        i = i * 2;
    }
}
```

2. Multiplying two $n \times n$ square matrices $A = (a_{ij})_{1 \leq i, j \leq n}$ and $B = (b_{ij})_{1 \leq i, j \leq n}$ gives a matrix,

$$C = \begin{pmatrix} c_{11} & c_{12} & \dots & c_{1n} \\ c_{21} & c_{22} & \dots & c_{2n} \\ \dots & \dots & \dots & \dots \\ c_{n1} & c_{n2} & \dots & c_{nn} \end{pmatrix}$$

wherein
$$c_{ij} = \sum_{k=1}^n a_{ik}b_{kj} \quad \forall 1 \leq i, j \leq n$$

The following pseudo-code multiplies two square matrices A and B to get matrix C:

```
for i from 1 to n          % loop i
  for j from 1 to n        % loop j
    C[i][j] = 0;
    for k from 1 to n      % loop k
      C[i][j] = C[i][j] + A[i][k] * B[k][j];
    end for
  end for
end for
```

Please give its asymptotic notation $T(n) \in O(g(n))$ (i.e $T(n)$ is Big-Oh of $g(n)$).

3. Compute Big-Oh notation.

```
a) void func(int n){  
    int i = 0;  
    while(i * i * i <= n)  
        i++; }
```

```
b) void recursive(int n, int m, int o) {  
    if (n <= 0) {  
        printf("%d, %d\n", m, o);  
    } else {  
        recursive(n - 1, m + 1, o);  
        recursive(n - 1, m, o + 1); } }
```

```
c) void func(int n) {  
    for (int i = 1, s = 0; i <= n; ++i) {  
        int t = 1;  
        for (int j = 1; j <= i; ++j)  
            t = t * j; } }
```

4. State whether the following statement is true or false (given n is the size of input), and give a brief justification for your answer.

a) If $f(n)$ and $g(n)$ are both $O(h(n))$, then $f(n)+g(n)$ is $O(h(n))$.

b) If $f(n) = \sum_{i=1}^n \log i$, then $f(n)$ is $O(\log n)$

5. Consider the following code fragment.

```
for i=1 to n do  
    for j=i to 2*i do  
        output hello
```

Let $T(n)$ denote the number of times 'hello' is printed as a function of n .

1) Express $T(n)$ as a summation;

2) Simplify the summation and give the worst-case running time using Big-Oh notation.